

[Accueil](#) / [Articles](#) / [CV](#) / [English](#) / [RSS](#)[Tout](#) / [Tech](#) / [Réflexions](#)

Publié le **20-06-2021** par **Vincent Jousse**

Dernière mise à jour Le **11-02-2024**

Tags [#web](#) [#python](#) [#framework](#) [#fastapi-tutorial](#) [#fastapi](#)

Le guide complet du débutant avec FastAPI - Partie 2 : templates html, base de données et documentation

J'ai toujours aimé apprendre par l'exemple et ce guide ne dérogera pas à la règle. Nous allons prendre comme prétexte la création d'un projet pour apprendre à nous servir de FastAPI. Nous allons développer une application de [publication de contenu/newsletter à la Substack](#).

Mise à jour Le 11/02/2024 : Tortoise n'étant pas activement maintenu, j'ai décidé de passer Le tutorial de Tortoise ORM à SQL Alchemy

L'intégralité du guide est [disponible ici](#).

Projet : une newsletter à la Substack

Les principales fonctionnalités que nous développerons :

- » Création d'articles
- » Envoi des articles par email
- » Gestion des utilisateurs avec inscription et authentification

Plein de bonus possibles :

- » Gestion du multilingue

- » Traduction automatique des articles
- » Commentaires sur les articles
- » Conversion du contenu en audio

Structure de notre projet FastAPI

À la différence de beaucoup de Framework, FastAPI n'impose **aucune structure de répertoires** ou de fichiers pour pouvoir fonctionner. Quelques **conventions** se dégagent cependant parmi tous les projets disponibles. Voici celle que nous allons adopter :

code

```
fastapi-beginners-guide/  <-- répertoire racine de notre projet
├─ app/                  <-- répertoire contenant le code Python
│   ├─ core/             <-- fichiers partagés (config, exceptions, ...)
│   │   └─ __init__.py
│   ├─ crud/             <-- création, récupération, mises à jour des
données
│   │   └─ __init__.py
│   ├─ __init__.py
│   ├─ main.py           <-- point d'entrée de notre programme FastAPI
│   ├─ models/           <-- les modèles de notre base de données
│   │   └─ __init__.py
│   ├─ schemas/          <-- les schémas de validation des modèles
│   │   └─ __init__.py
│   ├─ templates/        <-- fichiers html/jinja
│   ├─ tests/            <-- tests
│   │   └─ __init__.py
│   └─ views/            <-- fonctions gérant les requêtes HTTP
│       └─ __init__.py
├─ public/               <-- fichiers CSS, Javascript et fichiers
statiques
└─ venv/                 <-- environnement virtuel créé à la partie 1
```

Créez une structure de répertoire identique à celle ci-dessus. Vous ne devriez pas avoir à créer le répertoire `venv` puisque il a du être créé automatiquement suite à la [partie 1](#). Les fichiers `__init__.py` sont des fichiers vides nécessaires pour que Python puisse considérer vos répertoires comme des packages.

Copie/collez le code de la partie 1 dans le fichier `app/main.py` :

python

```
# app/main.py
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
async def root():
    return {"message": "Hello World"}
```

Déplacez vous à la racine du projet puis activez l'environnement virtuel :

bash

```
$ source ./venv/bin/activate
```

Assurez-vous ensuite que vous pouvez lancer uvicorn avec la commande suivante :

bash

```
(venv) $ uvicorn app.main:app --reload
```

Le type de résultat attendu :

log

```
INFO:      Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:      Started reloader process [10884] using watchgod
INFO:      Started server process [10886]
INFO:      Waiting for application startup.
INFO:      Application startup complete.
```

Notez la différence avec la commande de la [partie 1](#) : nous avons rajouté un `app.` devant le `main`. Celui-ci correspond au répertoire `app/` que nous venons de créer dans lequel se situe notre fichier `main.py`. Je rappelle que le `app` situé après le `:` est le nom de l'objet `FastAPI` qui a été créé dans notre fichier `main.py`.

Première page HTML

Nous allons maintenant créer la première page de notre « site web » avec de l'HTML et du CSS.

Copiez collez le code ci-dessous dans votre fichier `app/main.py`.

python

```
# app/main.py
from fastapi import FastAPI, Request
from fastapi.staticfiles import StaticFiles
from fastapi.templating import Jinja2Templates

app = FastAPI()

app.mount("/static", StaticFiles(directory="public"), name="public")

templates = Jinja2Templates(directory="app/templates")

@app.get("/")
async def root(request: Request):
    return templates.TemplateResponse(request, "home.html")
```

Décortiquons ce que nous venons de faire.

python

```
app.mount("/static", StaticFiles(directory="public"), name="public")
```

Nous « montons » (`app.mount`) une route qui va répondre à l'URL `/static` et qui servira, sous cette adresse, les fichiers que nous mettrons dans le répertoire `public/` précédemment créé (`directory="public"`). Nous nommons cette route `public` (`name="public"`), car nous aurons besoin de l'appeler par son nom pour nous en servir un peu plus loin. Toto aurait aussi fonctionné comme nom, mais c'est moins parlant 😊

En résumé : Si nous plaçons un fichier nommé `styles.css` dans notre répertoire `public/`, cette route va nous permettre d'y accéder par l'adresse `http://localhost:8000/public/styles.css`.

python

```
templates = Jinja2Templates(directory="app/templates")
```

Nous créons un objet (templates) qui va nous permettre de créer de l'HTML avec le moteur de templates [Jinja2](#). Cet objet ira chercher ses templates dans le répertoire que nous avons créé, app/templates/.

python

```
@app.get("/")
async def root(request: Request):
    return templates.TemplateResponse(request, "home.html")
```

Nous avons ensuite modifié notre méthode root pour qu'elle récupère l'objet request. Cet objet est fourni par FastAPI (plus précisément par [starlette](#), Framework sur lequel FastAPI est basé) et permet d'obtenir des informations sur la requête : l'URL d'origine, les cookies, les headers, etc. La documentation complète est disponible ici : <https://www.starlette.io/requests/>. Nous y reviendrons.

Au lieu de retourner un simple dictionnaire Python comme précédemment, notre méthode renvoie maintenant un objet TemplateResponse. C'est un objet qui va être en charge de créer du HTML à partir d'un template, home.html dans notre cas. Il ira chercher ce template dans le répertoire que nous avons spécifié plus haut avec directory="app/templates".

Il nous faut ensuite créer le contenu du template dans app/templates/home.html. Copiez-collez le code suivant :

jinja

```
<!DOCTYPE html>
<html>
<head>
    <title>Home</title>
    <link href="{{ url_for('public', path='/styles.css') }}"
rel="stylesheet">
</head>
<body>
    <h1>Home title</h1>
    <p>Current url: <strong>{{ request.url }}</strong></p>
```

```
</body>
</html>
```

Outre le code HTML classique, la première ligne intéressante est la suivante :

jinja

```
<link href="{{ url_for('public', path='/styles.css') }}" rel="stylesheet"
```

Nous construisons un lien dynamique grâce à la fonction `url_for`. Cette fonction prend en paramètres le nom de la route, `public` dans notre cas (le nom que nous avons donné plus haut, lors du `app.mount`) et l'emplacement du fichier, `path='/styles.css'`, qu'il nous restera à créer. Avec Jinja, tout ce qui est entre `{{` et `}}` sera affiché dans le code HTML.

L'autre ligne intéressante est celle-ci :

jinja

```
<p>Current url: <strong>{{ request.url }}</strong></p>
```

Ici nous nous servons de l'objet `request` de starlette que nous avons passé à notre template (`templates.TemplateResponse(request, "home.html")`) pour afficher l'url courante.

Il nous reste à créer le fichier `styles.css` dans le répertoire `public/` et d'y mettre le contenu suivant par exemple :

CSS

```
h1 {
    color: #fe5186;
}
```

Rechargez votre page d'accueil à l'adresse <http://localhost:8000/> et vous devriez obtenir le résultat suivant :



Interaction avec la base de données : écriture des modèles avec SQLAlchemy 2.0

Maintenant que nous arrivons à afficher quelque chose, il est temps de passer à la création de nos **modèles de base de données**. Ces modèles sont des classes spéciales Python qui vont nous aider à créer/modifier/supprimer des lignes dans la base de données.

Il y a plusieurs façon d'interagir avec une base de données. La façon classique est d'écrire des **requêtes SQL** directement par vous-même en fabricant vos propres `SELECT * FROM ...` et autres `UPDATE ... SET ...`. C'est faisable, mais ce n'est pas ce que l'on voit le plus souvent et c'est assez fastidieux. Je vais ici vous présenter une autre approche : l'utilisation d'un *Object Relational Mapper* (ORM). C'est ce que vous verrez dans quasiment tous les frameworks. Je ne rentrerai pas ici dans le débat sur l'efficacité ou non des ORM (car débat il y a) et j'adopterai juste une approche pragmatique : c'est ce que la majorité utilise, nous ferons donc pareil ici.

Pour faire simple, les ORMs vont vous permettre de faire du SQL et de créer vos propres requêtes SQL **sans écrire une ligne de SQL**, juste en manipulant des objets Python classiques.

Il existe beaucoup d'ORMs différents en Python :

- » L'[ORM de Django](#) lui est spécifique et ne peut pas être facilement utilisé en dehors de Django

- » [SqlAlchemy](#) est l'ORM standard de Python utilisé un peu partout
- » [peewee](#) un ORM simple et donc facile à apprendre
- » [Tortoise ORM](#) est un ORM inspiré de Django mais qui utilise les dernières avancées de Python (comme FastAPI), notamment `asyncio`.

Mon choix s'est porté sur [SqlAlchemy](#) car c'est celui que vous serez amenés à rencontrer le plus souvent. Il vient (janvier 2023) d'être mis à jour en version 2.0, c'est cette version que nous utiliserons dans ce tutoriel. Attention si vous cherchez des exemples de code sur le net, la plupart des exemples utilise encore la syntaxe 1.0 (qui reste compatible avec la 2.0).

Pour les besoins de ce guide, nous allons pour l'instant utiliser une base de données simple qui ne nécessite pas d'autres logiciels à installer : [SQLite](#). Nous verrons plus tard lorsque nous passerons à *Docker* comment utiliser une base de données bien plus robuste, à savoir [PostgreSQL](#).

Installation de SQLAlchemy

Soyez bien certain d'avoir activé votre environnement virtuel :

```
bash
```

```
$ source ./venv/bin/activate
```

Puis installez SQLAlchemy.

```
bash
```

```
(venv) $ pip install sqlalchemy
```

Création du modèle Article

Nous allons ajouter un premier modèle à notre application. Ce modèle va représenter un article dans notre Newsletter. Il aura donc les champs classiques auxquels on pourrait s'attendre : titre, contenu, etc.

Mettons à jour notre fichier `main.py` dans ce sens.

python

```
# app/main.py

from fastapi import FastAPI, Request
from fastapi.staticfiles import StaticFiles
from fastapi.templating import Jinja2Templates
from sqlalchemy import Column, DateTime, Integer, String, create_engine
from sqlalchemy.orm import declarative_base, sessionmaker
from sqlalchemy.sql import func

app = FastAPI()

app.mount("/public", StaticFiles(directory="public"), name="public")

templates = Jinja2Templates(directory="app/templates")

Base = declarative_base()

class Article(Base):
    __tablename__ = "articles"

    id = Column(Integer, primary_key=True)
    title = Column(String)
    content = Column(String)

    created_at = Column(String, server_default=func.now())
    updated_at = Column(DateTime, server_default=func.now(),
onupdate=func.now())

    def __str__(self):
        return self.title

@app.get("/")
async def root(request: Request):
    return templates.TemplateResponse(request, "home.html")
```

Tout d'abord, nous importons les classes nécessaires à SQLAlchemy :

python

```
from sqlalchemy import Column, DateTime, Integer, String, create_engine
from sqlalchemy.orm import declarative_base, sessionmaker
from sqlalchemy.sql import func
```

Ensuite, nous créons la classe de Base de SQLAlchemy qui va permettre la définition de tous nos modèles ensuite.

python

```
Base = declarative_base()
```

Puis nous définissons notre modèle, que nous allons nommer Article et qui hérite de la classe de base (Base) de SQLAlchemy :

python

```
class Article(Base):
```

Nous déclarons ici la clé primaire de notre modèle, de type Integer. Le `primary_key=True` va permettre de considérer le champ comme clé primaire et va générer la prochaine valeur automatiquement de manière incrémentale. Ce n'est pas quelque chose d'obligatoire puisque si nous ne le faisons pas.

python

```
id = Column(Integer, primary_key=True)
```

Nous déclarons ensuite nos champs de contenu, qui sont tous les deux de type String.

python

```
title = Column(String)
content = Column(String)
```

Je trouve toujours utile d'avoir la date de création de mes objets ainsi que leur dernière date de modification. Pour ce faire j'ai rajouté les deux champs `created_at` et `updated_at` qui seront

automatiquement mis à jour par la base de données. `server_default` permet de dire à la base de données d'exécuter une fonction à la création de l'objet. Dans notre cas, ça sera la fonction `now()` de la base de données qui retourne la date et l'heure courantes. `onupdate` permet de faire pareil, mais lors de la mise à jour de l'objet.

python

```
created_at = Column(String, server_default=func.now())
updated_at = Column(DateTime, server_default=func.now(),
onupdate=func.now())
```

Et pour finir, je surcharge la méthode Python par défaut `__str__`.

python

```
def __str__(self):
    return self.title
```

Il n'est pas obligatoire de surcharger la fonction `__str__` mais c'est une bonne pratique qui nous permettra de faciliter notre debug plus tard. Quand on demandera à afficher l'objet (plus précisément quand on aura besoin de sa représentation en tant que chaîne de caractères), cela affichera son titre au lieu d'une représentation incompréhensible interne à Python.

Il nous reste à déclarer notre base de données SQLAlchemy et à créer notre base de données, voici le code mis à jour pour ce faire :

python

```
# app/main.py
```

```
from fastapi import FastAPI, Request
from fastapi.staticfiles import StaticFiles
from fastapi.templating import Jinja2Templates
from sqlalchemy import Column, DateTime, Integer, String, create_engine
from sqlalchemy.orm import declarative_base, sessionmaker
from sqlalchemy.sql import func
```

```
app = FastAPI()
```

```
app.mount("/public", StaticFiles(directory="public"), name="public")
```

```
templates = Jinja2Templates(directory="app/templates")

SQLALCHEMY_DATABASE_URL = "sqlite:///./sql_app.db"

engine = create_engine(
    SQLALCHEMY_DATABASE_URL, connect_args={"check_same_thread": False}
)
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)

Base = declarative_base()

class Article(Base):
    __tablename__ = "articles"

    id = Column(Integer, primary_key=True)
    title = Column(String)
    content = Column(String)

    created_at = Column(String, server_default=func.now())
    updated_at = Column(DateTime, server_default=func.now(),
onupdate=func.now())

    def __str__(self):
        return self.title

Base.metadata.create_all(bind=engine)

@app.get("/")
async def root(request: Request):
    return templates.TemplateResponse(request, "home.html")
```

Nous créons l'engine SQLAlchemy en lui précisant l'emplacement du fichier de base de données `sql_app.db` et nous créons la classe `SessionLocal` qui nous permettra plus tard d'obtenir une connexion à la

base de données. Le paramètre `check_same_thread` est une particularité de SQLite vis à vis de FastAPI que nous ne détaillerons pas ici pour des raisons de simplicité.

Il ne faut pas oublier de créer les tables **APRÈS** la déclaration des modèles avec la ligne suivante :

python

```
Base.metadata.create_all(bind=engine)
```

Avec l'ajout du fichier de bases de données `sql_app.db` qui sera créé automatiquement dans notre projet, nous allons avoir besoin de changer la commande pour lancer uvicorn. En effet, le paramètre `--reload` de la commande uvicorn relance uvicorn à chaque fois qu'un fichier est modifié, peu importe où. Le problème est qu'à chaque fois que l'on modifiera la base de données uvicorn se rechargera automatiquement ce qui va finir par être pénible. Il suffit donc de spécifier à uvicorn où sont les fichiers qu'il doit surveiller pour s'auto-relancer avec le paramètre `--reload-dir` comme ci-dessous :

bash

```
(venv) $ uvicorn app.main:app --reload --reload-dir app
```

Il va maintenant nous rester à ajouter des méthodes pour créer et afficher nos articles.

Ajout d'un Article

Créons une méthode qui, à chaque fois que l'on appelle l'url `/articles/create`, va enregistrer un article dans la base de données. Évidemment, ce n'est pas optimal et nous changerons cette méthode un peu plus tard. Nous utiliserons notamment `@app.post` et non pas `@app.get` et nous créerons notre Article à partir d'un formulaire, mais pour un début, ça fera l'affaire.

Tout d'abord, mettez à jour les imports en ajoutant l'import de `Depends` au niveau de FastAPI et ajoutez la fonction `get_db` qui va vous permettre de créer une session d'accès à la base de données.

python

```
# app/main.py

from fastapi import Depends, FastAPI, Request
from fastapi.staticfiles import StaticFiles
from fastapi.templating import Jinja2Templates
from sqlalchemy import Column, DateTime, Integer, String, create_engine,
select
from sqlalchemy.orm import Session, declarative_base, sessionmaker
from sqlalchemy.sql import func

SQLALCHEMY_DATABASE_URL = "sqlite:///./sql_app.db"

engine = create_engine(
    SQLALCHEMY_DATABASE_URL, connect_args={"check_same_thread": False}
)
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)

Base = declarative_base()

app = FastAPI()

app.mount("/public", StaticFiles(directory="public"), name="public")

templates = Jinja2Templates(directory="app/templates")

# Dependency
def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

# ... reste du fichier
```

Puis ensuite ajoutez ces lignes au dessus de la fonction root :

```
python
```

```
# app/main.py

# ... début du fichier

@app.get("/articles/create")
async def articles_create(request: Request, db: Session = Depends(get_db)):
    article = Article(
        title="Mon titre de test", content="Un peu de contenu<br />avec
deux lignes"
    )
    db.add(article)
    db.commit()
    db.refresh(article)

    return templates.TemplateResponse(
        request, "articles_create.html", {"article": article}
    )

# ... reste du fichier
```

Commençons par nous arrêter à la définition de la fonction.

python

```
async def articles_create(request: Request, db: Session = Depends(get_db):
```

Ici FastAPI implémente un concept appelé [Injection de dépendances](#). L'idée est, via l'utilisation de `Depends`, d'injecter la connexion à la base de données à notre fonction. C'est FastAPI qui va se charger d'instancier cette connexion en faisant appel à la fonction `get_db`. L'injection de dépendances est un concept dont il est difficile de se passer une fois qu'on y a goûté : toutes vos dépendances sont explicites dans la signature de la fonction et en plus, vous n'avez pas à vous soucier de les créer, FastAPI le fait pour vous !

Pour plus d'informations, vous pouvez vous référer à la [documentation de FastAPI sur le sujet \(en anglais\)](#).

Ensuite nous instancions un objet `Article` puis nous demandons à la base de données de le créer via l'utilisation de la méthode `add`. Il

nous faut ensuite appeler `commit` sur notre connexion pour effectivement appliquer les changements à la base de données. Par défaut, les opérations sur la base de données sont réalisées dans ce que l'on appelle des transactions et ne sont enregistrées dans la base de données qu'à l'appel de la fonction `commit`.

Nous demandons ensuite à la base de données de « rafraîchir » l'objet article via l'utilisation de la fonction `refresh`. Cette fonction va, via une requête `SELECT` à la base de données, rafraîchir les champs de l'objet article. Dans notre cas cela va notamment permettre de mettre à jour l'id auto attribué par la base de données ainsi que les valeurs des champs `created_at` et `updated_at`.

Pour finir nous passons notre objet nouvellement créé à un template nommé `articles_create.html`. Notez le dictionnaire Python passé en troisième paramètre, `{"article": article}`. Ce dictionnaire va nous permettre de passer des données de notre vue à notre template. Dans ce cas précis, nous passons l'objet créé en paramètre. La clé du dictionnaire `"article"` est le nom que nous voulons donner à notre valeur dans le template et la valeur du dictionnaire `request` est la valeur que nous souhaitons passer au template sous ce nom.

Il vous reste à créer le template à l'emplacement `app/templates/articles_create.html` avec le contenu suivant :

jinja

```
<!DOCTYPE html>
<html>
<head>
    <title>Articles create</title>
    <link href="{% url_for('public', path='/styles.css') %}"
rel="stylesheet">
</head>
<body>
    <p>Article created: <strong>{% article %}</strong></p>
</body>
</html>
```



À chaque chargement de l'url <http://localhost:8000/articles/create> un objet sera créé dans la base de données et la page suivante devrait s'afficher :



Liste des articles : page HTML

Ajoutons un point d'entrée pour pouvoir afficher la liste de nos articles dans une page HTML. Ajoutez le code suivant juste avant la fonction root :

python

```
# app/main.py
```

```
# ... début du fichier
```

```
@app.get("/articles")
```

```
async def articles_list(request: Request, db: Session = Depends(get_db)):
```

```
    articles_statement = select(Article).order_by(Article.created_at)
```

```
    articles = db.scalars(articles_statement).all()
```

```
    return templates.TemplateResponse(
```

```
        request, "articles_list.html", {"articles": articles}
```

```
)
```

```
# ... fin du fichier
```

Dans une première partie, nous préparons notre expression sql (dans la variable `articles_statement`) en faisant un `select` sur la table `Article` et en triant les résultats par ordre croissant de création. Ensuite nous exécutons cette expression via l'utilisation de `db.scalars` et récupérons tous les résultats grâce à `all` (nous n'aurions pu récupérer que le premier résultat en utilisant `first` par exemple).

Pour finir nous passons notre variable `articles` sous le même nom à notre template `articles_list.html`.

Il nous reste maintenant à créer le template dans
app/templates/articles_list.html

jinja

```
<!DOCTYPE html>
<html>
<head>
    <title>Articles list</title>
    <link href="{{ url_for('public', path='/styles.css') }}"
rel="stylesheet">
</head>
<body>
    <h1>Liste des articles</h1>
    <ul>
    {% for article in articles %}
        <li>{{ article.id }} - {{ article.title }}</li>
    {% endfor %}
    </ul>
</body>
</html>
```

Notez la façon de réaliser des boucles avec le moteur de template Jinja2. Toute commande commence par un `{%` et finit par un `%}` sur la même ligne. Rien de bien sorcier à part le fait qu'il ne faut pas oublier de fermer le `for` avec `{% endfor %}`.

Ci-dessous le résultat que vous devriez avoir (au nombre d'articles prêt).



Liste des articles : API Json

Afficher du HTML c'est chouette et c'est la base du web. Mais comme je l'ai déjà mentionné en introduction, FastAPI est parfait pour réaliser des **API** (les parties cachées de vos applications mobiles notamment), et on aurait tort de s'en priver. Une Url d'API se comporte comme une URL web classique à la différence prêt qu'elle ne retourne généralement pas du contenu HTML mais juste **des données brutes**.

Notre premier *Hello World* était déjà une URL de *type API*, nous allons faire de même pour créer une API qui retourne la liste de nos articles. Placez le code suivant juste avant la fonction `root` :

```
python
```

```
# app/main.py

# ... début du contenu du fichier

@app.get("/api/articles")
async def api_articles_list(db: Session = Depends(get_db)):
    articles_statement = select(Article).order_by(Article.created_at)

    return db.scalars(articles_statement).all()

# ... fin du fichier
```

Et c'est aussi simple que ça. Au lieu de retourner un template comme nous le faisons jusqu'ici, nous retournons juste notre liste d'objets Python. FastAPI se charge de faire le reste.

Si vous vous rendez sur <http://localhost:8000/api/articles> dans votre navigateur, vous devriez voir s'afficher quelque chose comme cela :



```
[{"title": "Mon titre de test", "content": "Un peu de contenu<br />avec deux lignes", "updated_at": "2021-06-17T09:37:19.658747+00:00", "created_at": "2021-06-17T09:41:55.902871+00:00", "id": 1}, {"title": "Mon titre de test", "content": "Un peu de contenu<br />avec deux lignes", "updated_at": "2021-06-17T09:41:55.902871+00:00", "created_at": "2021-06-17T09:41:55.902766+00:00", "id": 2}, {"title": "Mon titre de test", "content": "Un peu de contenu<br />avec deux lignes", "updated_at": "2021-06-17T09:41:55.902871+00:00", "created_at": "2021-06-17T09:41:55.902766+00:00", "id": 3}]
```

Vous avouerez que ce n'est pas très sexy et pour cause, c'est juste de la donnée brute, sans formatage ou autre. Dans ce cas, notre navigateur Web classique ne sert pas à grand chose. Pour travailler

avec des API, il existe des outils spécialisés pour cela, dont un qui est écrit en Python, [httpie](https://httpie.dev/). Il est parfait pour ce que nous aurons à faire et je l'utiliserai comme référence à partir de maintenant.

Activez votre virtualenv et installez httpie :

```
bash
```

```
(venv) $ pip install httpie
```

Vous devriez ensuite pouvoir appeler la commande http (dans votre virtualenv) :

```
bash
```

```
(venv) $ http http://localhost:8000/api/articles
```

Et obtenir un résultat qui se rapproche de la capture d'écran ci-dessous :

```
→ http http://localhost:8000/api/articles
HTTP/1.1 200 OK
content-length: 478
content-type: application/json
date: Sun, 11 Feb 2024 15:08:36 GMT
server: uvicorn

[
  {
    "content": "Un peu de contenu<br />avec deux lignes",
    "created_at": "2024-02-11 15:08:31",
    "id": 1,
    "title": "Mon titre de test",
    "updated_at": "2024-02-11T15:08:31"
  },
  {
    "content": "Un peu de contenu<br />avec deux lignes",
    "created_at": "2024-02-11 15:08:32",
    "id": 2,
    "title": "Mon titre de test",
    "updated_at": "2024-02-11T15:08:32"
  },
  {
    "content": "Un peu de contenu<br />avec deux lignes",
    "created_at": "2024-02-11 15:08:33",
    "id": 3,
    "title": "Mon titre de test",
    "updated_at": "2024-02-11T15:08:33"
  }
]
```

La première ligne nous rappelle que nous utilisons le protocole **HTTP** dans sa version **1.1** et que le serveur nous a renvoyé un code de **status 200**. Dans le protocole HTTP, ce code de status 200 signifie que tout s'est bien passé (d'où le **OK** ensuite).

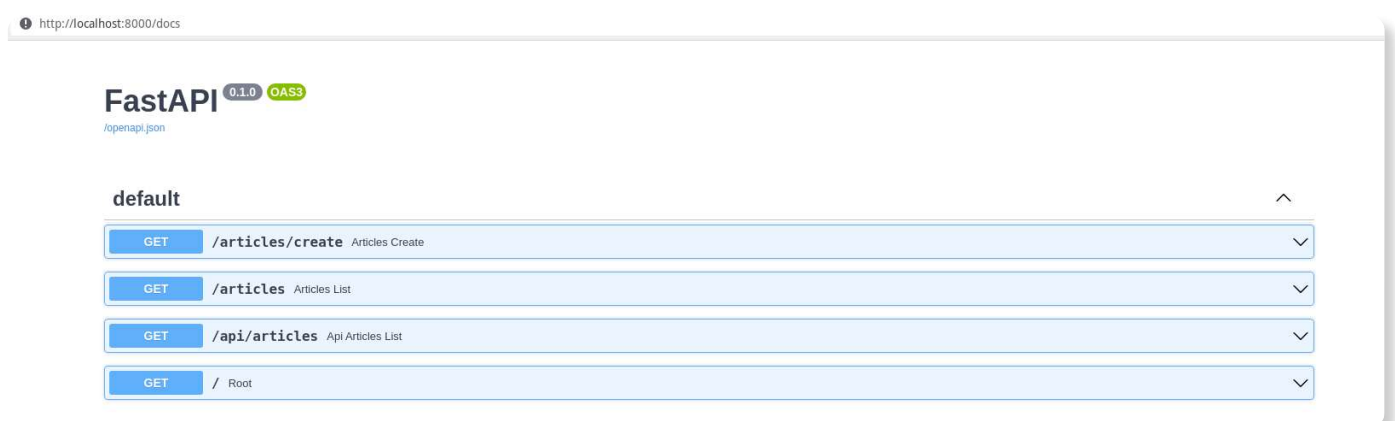
Les 4 lignes qui suivent sont ce que l'on appelle des entêtes (*headers* en anglais). Ce sont des informations qui viennent compléter la réponse envoyée par le serveur. Dans notre cas :

- » `content-length` : la taille de la réponse en octets.
- » `content-type` : le type de contenu renvoyé. Dans notre cas du json (`application/json`). On parle ici de type MIME (*MIME Types*).
- » `date` : la date et l'heure de la réponse.
- » `server` : le type de serveur qui a envoyé la réponse.

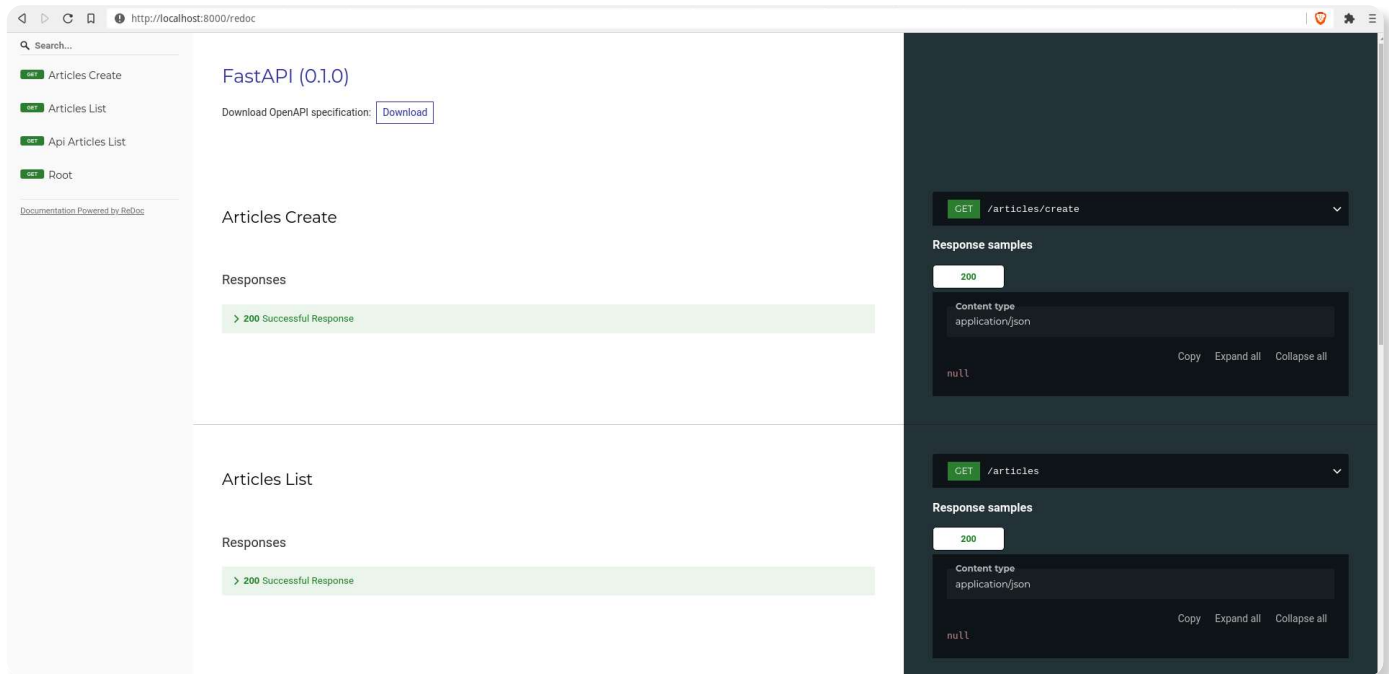
S'en vient ensuite le contenu de la réponse à proprement parler. Dans notre cas une liste (délimitée par `[` et `]`) d'objets json (délimités par `{` et `}`).

Documentation auto-générée

FastAPI est capable de générer la documentation de votre API automatiquement basé sur un standard nommé OpenAPI. Par défaut il génère une documentation avec Swagger à l'URL <http://localhost:8000/docs>



et une autre avec ReDoc à l'URL <http://localhost:8000/redoc>



Nous rentrerons un peu plus tard dans les détails de la documentation. Pour l'instant, nous allons juste faire en sorte de ne pas inclure, dans notre documentation d'API, les méthodes qui renvoient du HTML au lieu du JSON. En effet les méthode renvoyant du HTML sont celles qui sont destinées à afficher des pages dans le navigateur et non à renvoyer des données JSON via une API.

Pour cela nous allons modifier les décorateurs `@get` des fonctions que nous ne voulons pas voir apparaître dans la documentation en ajoutant un paramètre `include_in_schema=False`.

```
python
```

```
# app/main.py

# ...

@app.get("/articles/create", include_in_schema=False)
async def articles_create(request: Request):

# ...

@app.get("/articles", include_in_schema=False)
async def articles_list(request: Request):

# ...

@app.get("/", include_in_schema=False)
```

```
async def root(request: Request):
```

```
# ...
```

Il ne devrait plus rester que la fonction `/api/articles` dans votre documentation :

http://localhost:8000/docs#/default/api_articles_list_api_articles_get

FastAPI 0.1.0 0AS3
/openapi.json

default

GET

/api/articles

Api Articles List

Try it out

Parameters

No parameters

Responses

Code	Description	Links
200	Successful Response	No links

Media type

application/json

Controls Accept header:

Example Value | Schema

"string"

Conclusion

Nous venons de voir comment afficher du contenu HTML, connecter une base de données, développer un point d'entrée pour notre API Json et comment accéder à la documentation auto-générée. Les bases d'un projet FastAPI sont maintenant posées.

La prochaine étape va consister à réorganiser notre code pour qu'il puisse grossir un peu plus facilement. En effet, mettre tout notre code dans `main.py` va vite être ingérable. Nous verrons aussi comment mettre en place un début de tests automatisés.

Comme d'habitude, le code pour cette partie est [accessible directement sur Github](#).

Pour la partie 3, c'est par ici : [réorganisation du code, tests automatisés](#).

 Me contacter sur [Mastodon](#)

 Me contacter par [mail](#)

 S'abonner au flux [RSS](#)

 Recevoir les nouveaux articles par mail

(votre adresse ne sera jamais partagée à des tiers)

 Contenu fourni sous license [CC-By Licence](#).

 Construit avec [Rust](#), code source disponible sur [Github](#).

 Mettez-vous à Vim avec [mon livre à prix libre](#).